

Welcome to AddisCoder!

You will be learning how to write *code* in week 1. Code are instructions for the computer: the computer will do exactly what you tell it to do.

There are many ways to instruct the computer what to do. We will be using the programming language *Python*. Although programming languages are called "languages" they ARE NOT the same as a natural language, like Amharic or English.

We will be working in an environment called *Jupyter notebook*, which runs on Python.

How to use Jupyter

We will explore complex uses of Python in future lectures. However, for the time being, we will think of Jupyter as a very powerful calculator.

In Jupyter, each computation is performed in a *cell*. We'll demonstrate how to use cells now

```
1 + 1
2
4738 * 2357
11167466
738 * 3172
2340936
264 - 2381
-2117
```

Note on multiplication:

As we've seen, multiplication is done with `*`. It is NOT done with `x`. This is a common point of confusion. Please keep this in mind

Important note: shift + enter

In order to run a cell, you must press *shift + enter*. For the time being, only the last line in the cell will be output

```
1 + 1
2 + 2
1 + 1

2

432 - 321

111
```

Division is special

We've shown how to use addition, subtraction, and multiplication.

However, division is special.

```
1 / 2
0.5
-3/1
-3.0
43142 / -123821
-0.3484223193157865
1 / 3
0.3333333333333333
2 / 3
0.6666666666666666
10 // 3
3
1 // 2
0
-3 // 2
-2
-5 // 2
-3
```

```
-10 // 3  
-4
```

Order of operations

Operations happen in a specific order in Python. For the operations we've seen, they happen in the following order:

- Multiplication/Division
- Addition/Subtraction

Note that multiplication and division have *equal* precedence. They are simply executed from left to right. Let's see some examples

```
1 * 2 + 3  
5  
1 + 2 * 3  
7  
1 + 2 * 3 // 2  
4  
1 + 2 * 3 // 2 - 3  
1
```

Order of operations: parentheses

You can also use parentheses in Python. Parentheses always happen first

```
2 * 2 + 3  
7  
2 * (2 + 3)  
10
```

Special operation: exponentiation

Suppose you want to compute 2^5 . You could write $2 * 2 * 2 * 2 * 2$, but this is quite verbose!

We can directly exponentiate in Python.

The syntax for exponentiation in Python is `a ** b` for a^b . It is NOT `^`! Beware

```
2 * 2 * 2 * 2 * 2
```

```
32
```

```
2 ** 5
```

```
32
```

```
321 ** 3
```

```
33076161
```

```
4 ** 0.5
```

```
2.0
```

```
4 ** -1
```

```
0.25
```

Special operation: modulo

There is a special operation called the *modulo* operation. It takes the remainder after division.

As an example, consider `5 // 2`, which is equal to 2. The *remainder* of `5 // 2` is 1. So, the modulus of 5 divided by 2 is 1.

The syntax in Python for modulus is `%`. We'll see some examples.

```
5 // 2
```

```
2
```

```
5 % 2
```

```
1
```

```
5 - 2 * 2
```

```
1
```

```
10 % 2
```

```
0
```

```
2 % 8
```

```
2
```

```
2 // 8
```

```
0
```

```
2 - 8 * 0
```

```
2
```

```
10 % 8 % 6
```

```
2
```

```
10 % 8
```

```
2
```

```
2 % 6
```

```
2
```

Order of operations: putting it all together

The order of operations with all of the operations is:

1. Parentheses `()`
2. Exponent `**`
3. Multiplication `*` and Division `/` `//` `%`, left to right
4. Addition `+` and Subtraction `-`, left to right

Everything in the same line has equal precedence!

```
(1 + 2) * 3 ** 2 // 2
```

```
13
```

```
(3) * 3 ** 2 // 2
```

```
13
```

```
(3) * 9 // 2
```

```
13
```

```
27 // 2
```

```
13
```

```
5 // 2 * 4
```

```
8
```

```
5 * 2 // 4
2
6 // 5 * 8 / 3
2.6666666666666665
```

Errors

Programming languages, unlike natural languages, are *very specific*. If you don't write your code precisely, you will get an *error*.

We'll now see some examples of errors. You'll see more errors in lab!

```
2 * * 4
Cell In[90], line 1
  2 * * 4
    ^
SyntaxError: invalid syntax
```

```
1 +
Cell In[91], line 1
  1 +
    ^
SyntaxError: invalid syntax
```

```
2 * * 4
Cell In[92], line 1
  2 * * 4
    ^
SyntaxError: invalid syntax
```

```
10 / 0
-----
-----
ZeroDivisionError                                Traceback (most recent call
last)
Cell In[93], line 1
----> 1 10 / 0

ZeroDivisionError: division by zero
```

```
3 x 4
```

```
Cell In[31], line 1
```

```
3 x 4  
  ^
```

```
SyntaxError: invalid syntax
```

Comments

So far, we have written *code*. Code is what Python can understand. Code is for the computer.

Sometimes, we want to write *comments* that are for humans. Python ignores comments

```
# Nothing happens
```

```
# This is ignored
```

```
1 + 1
```

```
2
```

```
# This is ignored
```

```
1 + 1 # You can have comments inline!
```

```
2
```

What will happen?

```
# 1 + 1
```

If you're confused, test in Jupyter!

We have limited time in lecture. Most of the time in this course will be spent in lab. In many circumstances, you will explore questions yourself.

The following examples are confusing: the best way to find out is to test yourself!

```
10 % -1
```

```
0
```

```
10.2321 % 2
```

```
0.232100000000000086
```

```
20 ** 0.5
```

4.47213595499958

10 % 20

10